```
#include <stdio.h>
#include <stdlib.h>

////////////////////////////////////////////////
// 🔹 BST & AVL COMMON STRUCTURE
////////////////////////////////////////////////

struct Node {
    int data;
    struct Node *left, *right;
    int height; // used only for AVL
};

int max(int a, int b) { return (a > b) ? a : b; }

int height(struct Node* n) {
    return n ? n->height : 0;
}

struct Node* createNode(int data) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->data = data;
    n->left = n->right = NULL;
    n->height = 1;
    return n;
}

////////////////////////////////////////////////
// 🔹 BST FUNCTIONS
////////////////////////////////////////////////

struct Node* insertBST(struct Node* root, int data) {
    if (root == NULL) return createNode(data);

    if (data < root->data)
        root->left = insertBST(root->left, data);
    else
        root->right = insertBST(root->right, data);

    return root;
}

////////////////////////////////////////////////
// 🔹 AVL FUNCTIONS
```

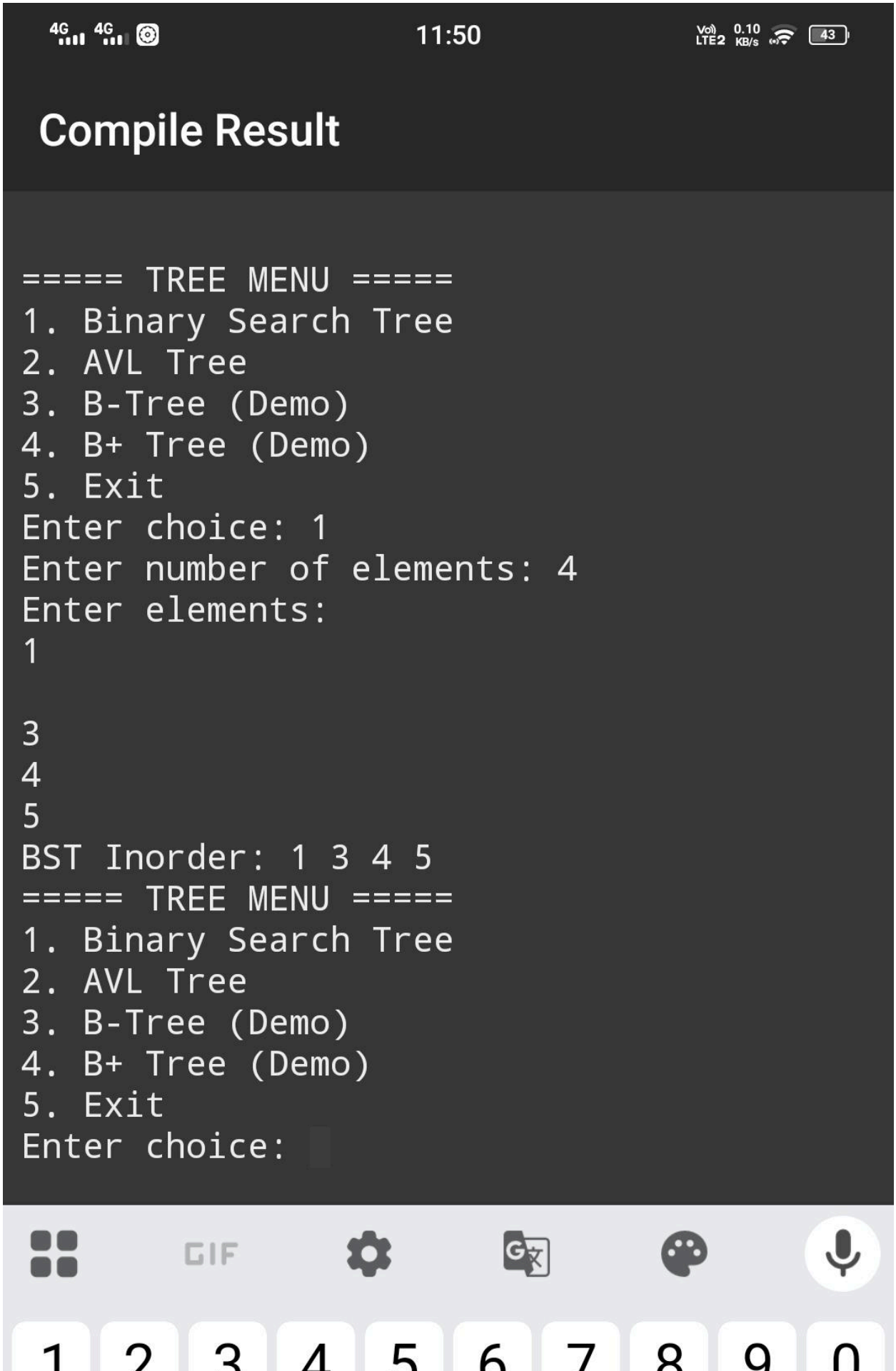
```
//////////////////////////////////////////////////

int getBalance(struct Node* n) {
    return n ? height(n->left) - height(n->right) : 0;
}

struct Node* rightRotate(struct Node* y) {
    struct Node* x = y->left;
    struct Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

struct Node* leftRotate(struct Node* x) {
    struct Node* y = x->right;
    struct Node* T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

struct Node* insertAVL(struct Node* node, int key) {
    if (node == NULL) return createNode(key);

    if (key < node->data)
        node->left = insertAVL(node->left, key);
    else if (key > node->data)
        node->right = insertAVL(node->right, key);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));
```

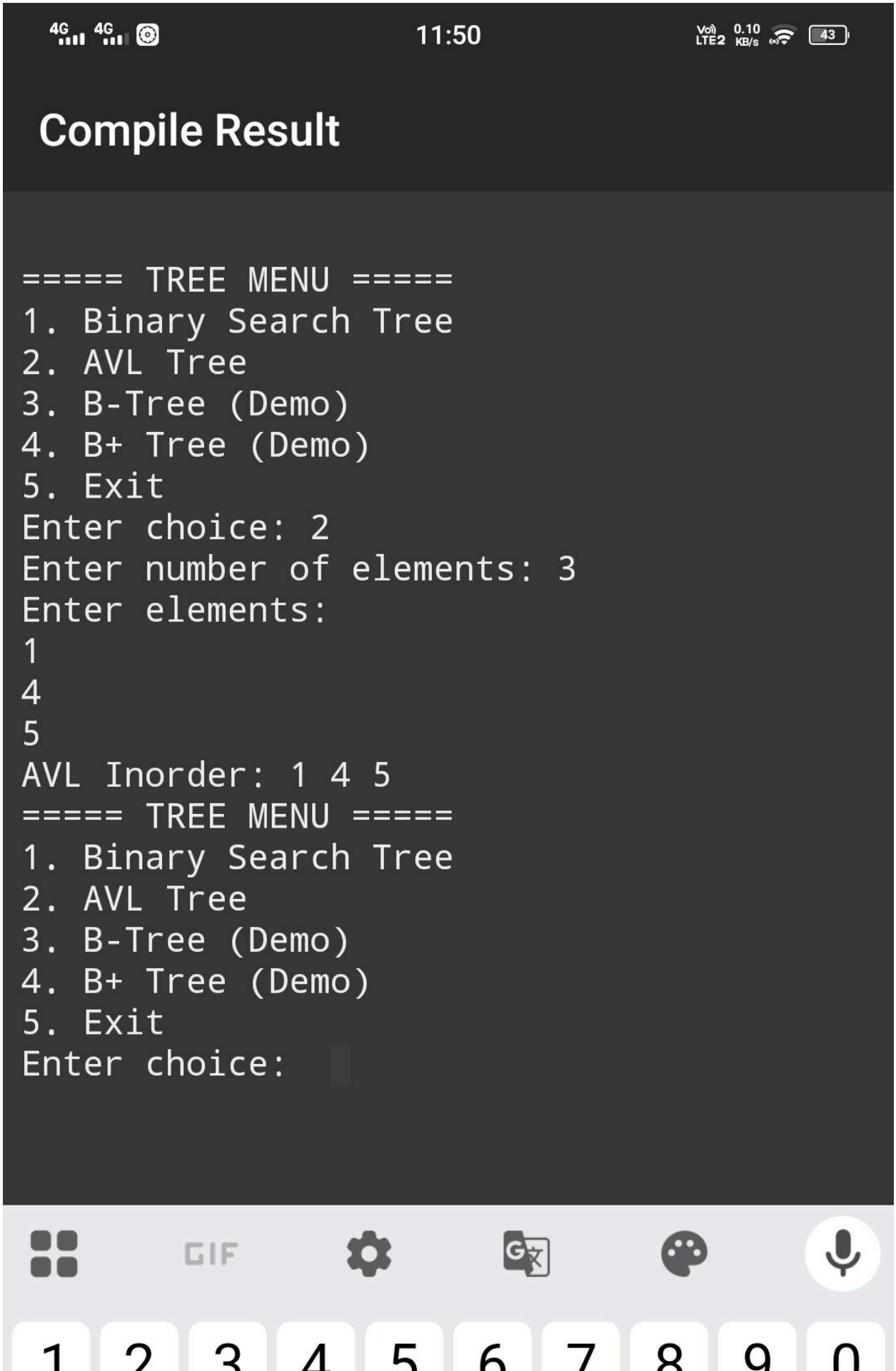
```
    int balance = getBalance(node);

    if (balance > 1 && key < node->left->data)
        return rightRotate(node);

    if (balance < -1 && key > node->right->data)
        return leftRotate(node);

    if (balance > 1 && key > node->left->data) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    if (balance < -1 && key < node->right->data) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

////////////////////////////////////////////////
// 🔹 SIMPLE B-TREE (DEMO)
////////////////////////////////////////////////

#define MAX 3

struct BTree {
    int val[MAX];
    int count;
};

void insertBTree(struct BTree* t, int val) {
    if (t->count < MAX) {
        t->val[t->count++] = val;
    } else {
        printf("B-Tree node full (demo only)\n");
    }
}

void displayBTree(struct BTree* t) {
    for (int i = 0; i < t->count; i++)
        printf("%d ", t->val[i]);
}
```

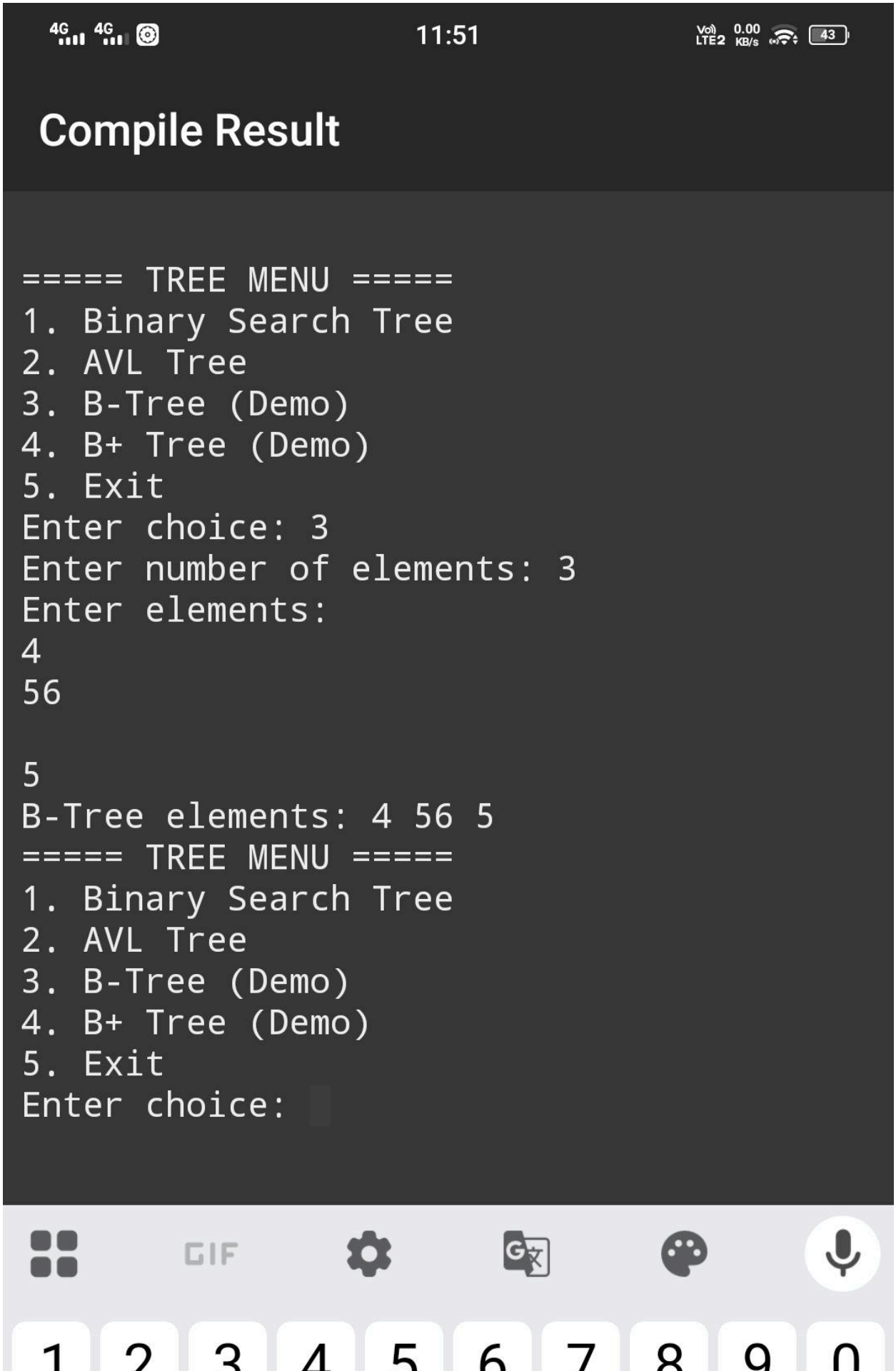
```
////////////////////////////////////////////////
//  🔹 SIMPLE B+ TREE (DEMO)
////////////////////////////////////////////////

struct BPlus {
    int keys[MAX];
    int count;
};

void insertBPlus(struct BPlus* t, int val) {
    if (t->count < MAX) {
        t->keys[t->count++] = val;
    } else {
        printf("B+ Tree node full (demo only)\n");
    }
}

void displayBPlus(struct BPlus* t) {
    for (int i = 0; i < t->count; i++)
        printf("%d ", t->keys[i]);
}

////////////////////////////////////////////////
//  🔹 COMMON TRAVERSAL
////////////////////////////////////////////////

void inorder(struct Node* root) {
    if (root) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

////////////////////////////////////////////////
//  🔹 MAIN MENU
////////////////////////////////////////////////

int main() {
    int choice, val, n;

    struct Node* bstRoot = NULL;
    struct Node* avlRoot = NULL;
```

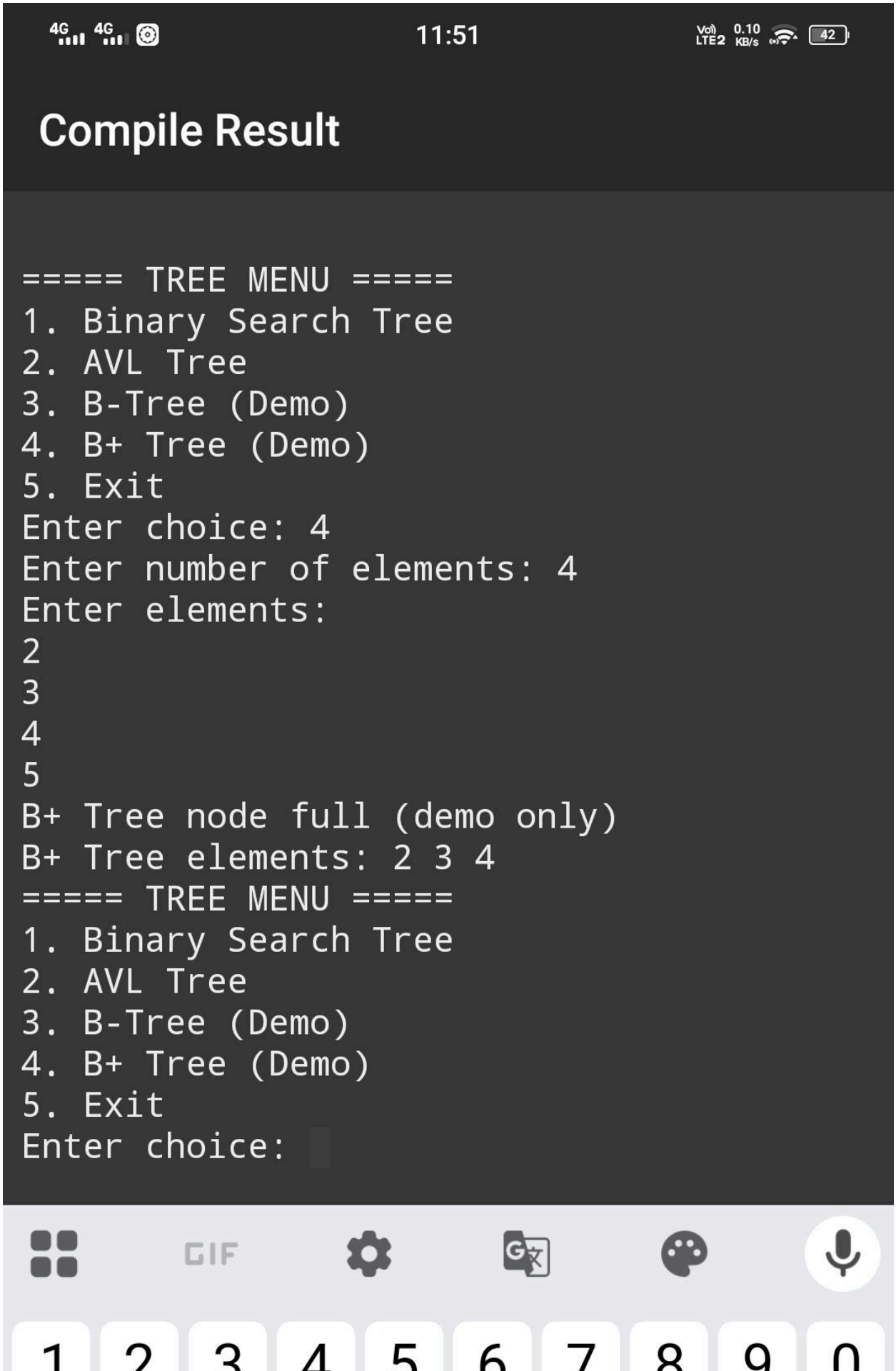
```
struct BTree bt = {.count = 0};
struct BPlus bpt = {.count = 0};

while (1) {
    printf("\n===== TREE MENU =====\n");
    printf("1. Binary Search Tree\n");
    printf("2. AVL Tree\n");
    printf("3. B-Tree (Demo)\n");
    printf("4. B+ Tree (Demo)\n");
    printf("5. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 5) break;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    switch (choice) {

    case 1:
        bstRoot = NULL;
        printf("Enter elements:\n");
        for (int i = 0; i < n; i++) {
            scanf("%d", &val);
            bstRoot = insertBST(bstRoot, val);
        }
        printf("BST Inorder: ");
        inorder(bstRoot);
        break;

    case 2:
        avlRoot = NULL;
        printf("Enter elements:\n");
        for (int i = 0; i < n; i++) {
            scanf("%d", &val);
            avlRoot = insertAVL(avlRoot, val);
        }
        printf("AVL Inorder: ");
        inorder(avlRoot);
        break;

    case 3:
        bt.count = 0;
```

```
            printf("Enter elements:\n");
            for (int i = 0; i < n; i++) {
                scanf("%d", &val);
                insertBTree(&bt, val);
            }
            printf("B-Tree elements: ");
            displayBTree(&bt);
            break;

        case 4:
            bpt.count = 0;
            printf("Enter elements:\n");
            for (int i = 0; i < n; i++) {
                scanf("%d", &val);
                insertBPlus(&bpt, val);
            }
            printf("B+ Tree elements: ");
            displayBPlus(&bpt);
            break;

        default:
            printf("Invalid choice!\n");
        }
    }

    return 0;
}
```


11:50
Compile Result
===== TREE MENU =====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Demo)
4. B+ Tree (Demo)
5. Exit
Enter choice: 1
Enter number of elements: 4
Enter elements:
1
3
4
5
BST Inorder: 1 3 4 5
===== TREE MENU =====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Demo)
4. B+ Tree (Demo)
5. Exit
Enter choice:
GIF
1 2 3 4 5 6 7 8 9 0


11:50
Compile Result
===== TREE MENU =====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Demo)
4. B+ Tree (Demo)
5. Exit
Enter choice: 2
Enter number of elements: 3
Enter elements:
1
4
5
AVL Inorder: 1 4 5
===== TREE MENU =====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Demo)
4. B+ Tree (Demo)
5. Exit
Enter choice:
GIF
1 2 3 4 5 6 7 8 9 0


11:51
Compile Result
===== TREE MENU =====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Demo)
4. B+ Tree (Demo)
5. Exit
Enter choice: 3
Enter number of elements: 3
Enter elements:
4
56
5
B-Tree elements: 4 56 5
===== TREE MENU =====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Demo)
4. B+ Tree (Demo)
5. Exit
Enter choice:
GIF
1 2 3 4 5 6 7 8 9 0


11:51
Compile Result
===== TREE MENU =====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Demo)
4. B+ Tree (Demo)
5. Exit
Enter choice: 4
Enter number of elements: 4
Enter elements:
2
3
4
5
B+ Tree node full (demo only)
B+ Tree elements: 2 3 4
===== TREE MENU =====
1. Binary Search Tree
2. AVL Tree
3. B-Tree (Demo)
4. B+ Tree (Demo)
5. Exit
Enter choice:
GIF
1 2 3 4 5 6 7 8 9 0